

An important application of the inorder traversal algorithm arises when we store an ordered sequence of elements in a binary tree, defining a structure we call a binary search tree. Let S be a set whose unique elements have an order relation. For example, S could be a set of integers. A binary search tree for S is a binary tree T such that, for each position p of T :

- Position p stores an element of S , denoted as $e(p)$.
- Elements stored in the left subtree of p (if any) are less than $e(p)$.
- Elements stored in the right subtree of p (if any) are greater than $e(p)$.

An example of a binary search tree is shown in Figure 8.19. The above properties assure that an inorder traversal of a binary search tree T visits the elements in nondecreasing order.

36

25

31

42

12

62

75

58

90

Figure 8.19: A binary search tree storing integers. The solid path is traversed when searching (successfully) for 36. The dashed path is traversed when searching (unsuccessfully) for 70.

We can use a binary search tree T for set S to find whether a given search value v is in S , by traversing a path down the tree T , starting at the root. At each internal position p encountered, we compare our search value v with the element $e(p)$ stored at p . If $v < e(p)$, then the search continues in the left subtree of p . If $v = e(p)$, then the search terminates successfully. If $v > e(p)$, then the search continues in the right subtree of p . Finally, if we reach an empty subtree, the search terminates unsuccessfully. In other words, a binary search tree can be viewed as a binary decision tree (recall Example 8.6), where the question asked at each internal node is whether the element at that node is less than, equal to, or larger than the element being searched for. We illustrate several examples of the search operation in Figure 8.19.

Note that the running time of searching in a binary search tree T is proportional to the height of T . Recall from Proposition 8.8 that the height of a binary tree with n nodes can be as small as $\log(n+1) \cdot 1$ or as large as $n \cdot 1$. Thus, binary search trees are most efficient when they have small height. Chapter 11 is devoted to the study of search trees.

Case Study: An Expression Tree

In Example 8.7, we introduced the use of a binary tree to represent the structure of an arithmetic expression. In this section, we define a new `ExpressionTree` class that provides support for constructing such trees, and for displaying and evaluating the arithmetic expression that such a tree represents. Our `ExpressionTree` class is defined as a subclass of `LinkedBinaryTree`, and we rely on the nonpublic mutators to construct such trees. Each internal node must store a string that defines a binary operator (e.g., `+`), and each leaf must store a numeric value (or a string representing a numeric value).

Our eventual goal is to build arbitrarily complex expression trees for compound arithmetic expressions such as $((3 + 1) \times 4) / ((9 \cdot 5) + 2)$. However, it suffices for the `ExpressionTree` class to support two basic forms of initialization:

`ExpressionTree(value)`: Create a tree storing the given value at the root.

`ExpressionTree(op, E1, E2)`: Create a tree storing string `op` at the root (e.g., `+`), and with the structures of existing `ExpressionTree` instances `E1` and `E2` as the left and right subtrees of the root, respectively.

Such a constructor for the `ExpressionTree` class is given in Code Fragment 8.35. The class formally inherits from `LinkedBinaryTree`, so it has access to all the nonpublic update methods that were defined in Section 8.3.1. We use `addRoot` to create an initial root of the tree storing the token provided as the first parameter. Then we perform run-time checking of the parameters to determine whether the caller invoked the one-parameter version of the constructor (in which case, we are done), or the three-parameter form. In that case, we use the inherited `attach` method to incorporate the structure of the existing trees as subtrees of the root.

Composing a Parenthesized String Representation

A string representation of an existing expression tree instance, for example, as $((3+1) \times 4) / ((9 \cdot 5) + 2)$, can be produced by displaying tree elements using an inorder traversal, but with opening and closing parentheses inserted with a preorder and postorder step, respectively. In the context of an `ExpressionTree` class, we support a special

`str`

method (see Section 2.3.2) that returns the appropriate string. Because it is more efficient to first build a sequence of individual strings to be joined together (see discussion of ‘Composing Strings’ in Section 5.4.2), the implementation of

`str`

relies on a nonpublic, recursive method named `parenthesize recur` that appends a series of strings to a list. These methods are included in Code 8.35.

11.1. Binary Search Trees

461

11.1.1

Navigating a Binary Search Tree

We begin by demonstrating that a binary search tree hierarchically represents the sorted order of its keys. In particular, the structural property regarding the placement of keys within a binary search tree assures the following important consequence regarding an inorder traversal (Section 8.4.3) of the tree.

Proposition 11.1: An inorder traversal of a binary search tree visits positions in increasing order of their keys.

Justification:

We prove this by induction on the size of a subtree. If a subtree has at most one item, its keys are trivially visited in order. More generally, an inorder traversal of a (sub)tree consists of a recursive traversal of the (possibly empty) left subtree, followed by a visit of the root, and then a recursive traversal of the (possibly empty) right subtree. By induction, a recursive inorder traversal of the left subtree will produce an iteration of the keys in that subtree in increasing order. Furthermore, by the binary search tree property, all keys in the left subtree have keys strictly smaller than that of the root. Therefore, visiting the root just after that subtree extends the increasing order of keys. Finally, by the search tree property, all keys in the right subtree are strictly greater than the root, and by induction, an inorder traversal of that subtree will visit those keys in increasing order.

Since an inorder traversal can be executed in linear time, a consequence of this proposition is that we can produce a sorted iteration of the keys of a map in linear time, when represented as a binary search tree.

Although an inorder traversal is typically expressed using a top-down recursion, we can provide nonrecursive descriptions of operations that allow more fine-grained navigation among the positions of a binary search relative to the order of their keys. Our generic binary tree ADT from Chapter 8 is defined as a positional structure, allowing direct navigation using methods such as `parent(p)`, `left(p)`, and `right(p)`. With a binary search tree, we can provide additional navigation based on the natural order of the keys stored in the tree. In particular, we can support the following methods, akin to those provided by a `PositionalList` (Section 7.4.1).

`first()`: Return the position containing the least key, or `None` if the tree is empty.

`last()`: Return the position containing the greatest key, or `None` if empty tree.

`before(p)`: Return the position containing the greatest key that is less than that of position `p` (i.e., the position that would be visited immediately before `p` in an inorder traversal), or `None` if `p` is the first position.

`after(p)`: Return the position containing the least key that is greater than that of position `p` (i.e., the position that would be visited immediately after `p` in an inorder traversal), or `None` if `p` is the last position.

Red-Black Trees

Although AVL trees and (2,4) trees have a number of nice properties, they also have some disadvantages. For instance, AVL trees may require many restructure operations (rotations) to be performed after a deletion, and (2,4) trees may require many split or fusing operations to be performed after an insertion or removal. The data structure we discuss in this section, the red-black tree, does not have these drawbacks; it uses $O(1)$ structural changes after an update in order to stay balanced. Formally, a red-black tree is a binary search tree (see Section 11.1) with nodes colored red and black in a way that satisfies the following properties:

Root Property: The root is black.

Red Property: The children of a red node (if any) are black.

Depth Property: All nodes with zero or one children have the same black depth, defined as the number of black ancestors. (Recall that a node is its own ancestor).

An example of a red-black tree is shown in Figure 11.30.

10
13
17
15
12
5
3
4
8
6
7
11
14

Figure 11.30: An example of a red-black tree, with red nodes drawn in white. The common black depth for this tree is 3.

We can make the red-black tree definition more intuitive by noting an interesting correspondence between red-black trees and (2,4) trees (excluding their trivial external nodes). Namely, given a red-black tree, we can construct a corresponding (2,4) tree by merging every red node w into its parent, storing the entry from w at its parent, and with the children of w becoming ordered children of the parent. For example, the red-black tree in Figure 11.30 corresponds to the (2,4) tree from Figure 11.24, as illustrated in Figure 11.31. The depth property of the red-black tree corresponds to the depth property of the (2,4) tree since exactly one black node of the red-black tree contributes to each node of the corresponding (2,4) tree.

Conversely, we can transform any (2,4) tree into a corresponding red-black tree by coloring each node w black and then performing the following transformations, as illustrated in Figure 11.32.

11.7. Exercises

529

R-11.13 The rules for a deletion in an AVL **tree** specifically require that when the two subtrees of the node denoted as y have equal height, child x should be chosen to be aligned with y (so that x and y are both left children or both right children). To better understand this requirement, repeat Exercise R-11.11 assuming we picked the misaligned choice of x . Why might there be a problem in restoring the AVL property with that choice?

R-11.14 Perform the following sequence of operations in an initially empty splay **tree** and draw the **tree** after each set of operations.

- Insert keys 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, in this order.
- Search for keys 1, 3, 5, 7, 9, 11, 13, 15, 17, 19, in this order.
- Delete keys 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, in this order.

R-11.15 What does a splay **tree** look like if its entries are accessed in increasing order by their keys?

R-11.16 Is the search **tree** of Figure 11.23(a) a (2,4) **tree**? Why or why not?

R-11.17 An alternative way of performing a split at a node w in a (2,4) **tree** is to partition w into $w_{\cdot}$ and $w_{\cdot\cdot}$, with $w_{\cdot}$ being a 2-node and $w_{\cdot\cdot}$ a 3-node.

Which of the keys k_1, k_2, k_3 , or k_4 do we store at w 's parent? Why?

R-11.18 Dr. Amongus claims that a (2,4) **tree** storing a set of entries will always have the same **structure**, regardless of the order in which the entries are inserted. Show that he is wrong.

R-11.19 Draw four different red-black **trees** that correspond to the same (2,4) **tree**.

R-11.20 Consider the set of keys $K = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15\}$.

- Draw a (2,4) **tree** storing K as its keys using the fewest number of nodes.
- Draw a (2,4) **tree** storing K as its keys using the maximum number of nodes.

R-11.21 Consider the sequence of keys (5,16,22,45,2,10,18,30,50,12,1). Draw the result of inserting entries with these keys (in the given order) into

- An initially empty (2,4) **tree**.
- An initially empty red-black **tree**.

R-11.22 For the following statements about red-black **trees**, provide a justification for each true statement and a counterexample for each false one.

- A subtree of a red-black **tree** is itself a red-black **tree**.
- A node that does not have a sibling is red.
- There is a unique (2,4) **tree** associated with a given red-black **tree**.
- There is a unique red-black **tree** associated with a given (2,4) **tree**.

R-11.23 Explain why you would get the same output in an inorder listing of the entries in a binary search **tree**, T , independent of whether T is maintained to be an AVL **tree**, splay **tree**, or red-black **tree**.

8.1. General Trees

305

8.1.2

The Tree Abstract Data Type

As we did with positional lists in Section 7.4, we define a tree ADT using the concept of a position as an abstraction for a node of a tree. An element is stored at each position, and positions satisfy parent-child relationships that define the tree structure. A position object for a tree supports the method:

p.element(): Return the element stored at position p.

The tree ADT then supports the following accessor methods, allowing a user to navigate the various positions of a tree:

T.root(): Return the position of the root of tree T,
or None if T is empty.

T.is_root(p): Return True if position p is the root of Tree T.

T.parent(p): Return the position of the parent of position p,
or None if p is the root of T.

T.num_children(p): Return the number of children of position p.

T.children(p): Generate an iteration of the children of position p.

T.is_leaf(p): Return True if position p does not have any children.

len(T): Return the number of positions (and hence elements) that
are contained in tree T.

T.is_empty(): Return True if tree T does not contain any positions.

T.positions(): Generate an iteration of all positions of tree T.

iter(T): Generate an iteration of all elements stored within tree T.

Any of the above methods that accepts a position as an argument should generate a ValueError if that position is invalid for T.

If a tree T is ordered, then T.children(p) reports the children of p in the natural order. If p is a leaf, then T.children(p) generates an empty iteration. In similar regard, if tree T is empty, then both T.positions() and iter(T) generate empty iterations. We will discuss general means for iterating through all positions of a tree in Sections 8.4.

We do not define any methods for creating or modifying trees at this point.

We prefer to describe different tree update methods in conjunction with specific implementations of the tree interface, and specific applications of trees.

Chapter 11. Search Trees

In the context of a tree-balancing algorithm, a rotation allows the shape of a tree to be modified while maintaining the search tree property. If used wisely, this operation can be performed to avoid highly unbalanced tree configurations. For example, a rightward rotation from the first formation of Figure 11.8 to the second reduces the depth of each node in subtree T1 by one, while increasing the depth of each node in subtree T3 by one. (Note that the depth of nodes in subtree T2 are unaffected by the rotation.)

One or more rotations can be combined to provide broader rebalancing within a tree. One such compound operation we consider is a trinode restructuring. For this manipulation, we consider a position x , its parent y , and its grandparent z . The goal is to restructure the subtree rooted at z in order to reduce the overall path length to x and its subtrees. Pseudo-code for a `restructure(x)` method is given in Code Fragment 11.9 and illustrated in Figure 11.9. In describing a trinode restructuring, we temporarily rename the positions x , y , and z as a , b , and c , so that a precedes b and b precedes c in an inorder traversal of T . There are four possible orientations mapping x , y , and z to a , b , and c , as shown in Figure 11.9, which are unified into one case by our relabeling. The trinode restructuring replaces z with the node identified as b , makes the children of this node be a and c , and makes the children of a and c be the four previous children of x , y , and z (other than x and y), while maintaining the inorder relationships of all the nodes in T .

Algorithm `restructure(x)`:

Input: A position x of a binary search tree T that has both a parent y and a grandparent z

Output: Tree T after a trinode restructuring (which corresponds to a single or double rotation) involving positions x , y , and z

- 1: Let (a, b, c) be a left-to-right (inorder) listing of the positions x , y , and z , and let (T_1, T_2, T_3, T_4) be a left-to-right (inorder) listing of the four subtrees of x , y , and z not rooted at x , y , or z .
- 2: Replace the subtree rooted at z with a new subtree rooted at b .
- 3: Let a be the left child of b and let T_1 and T_2 be the left and right subtrees of a , respectively.
- 4: Let c be the right child of b and let T_3 and T_4 be the left and right subtrees of c , respectively.

Code Fragment 11.9: The trinode restructuring operation in a binary search tree.

In practice, the modification of a tree T caused by a trinode restructuring operation can be implemented through case analysis either as a single rotation (as in Figure 11.9a and b) or as a double rotation (as in Figure 11.9c and d). The double rotation arises when position x has the middle of the three relevant keys and is first rotated above its parent, and then above what was originally its grandparent. In any of the cases, the trinode restructuring is completed with $O(1)$ running time.

Chapter 8. Trees

Python Implementation of a Linked Binary Tree Structure

In this section, we define a concrete `LinkedBinaryTree` class that implements the binary tree ADT by subclassing the `BinaryTree` class. Our general approach is very similar to what we used when developing the `PositionalList` in Section 7.4: We define a simple, nonpublic `Node` class to represent a node, and a public `Position` class that wraps a node. We provide a `validate` utility for robustly checking the validity of a given position instance when unwrapping it, and a `make_position`

utility for wrapping a node as a position to return to a caller.

Those definitions are provided in Code Fragment 8.8. As a formality, the new `Position` class is declared to inherit immediately from `BinaryTree.Position`. Technically, the `BinaryTree` class definition (see Code Fragment 8.7) does not formally declare such a nested class; it trivially inherits it from `Tree.Position`. A minor benefit from this design is that our position class inherits the

special method

so that syntax `p != q` is derived appropriately relative to

eq

Our class definition continues, in Code Fragment 8.9, with a constructor and with concrete implementations for the methods that remain abstract in the `Tree` and `BinaryTree` classes. The constructor creates an empty tree by initializing `root` to `None` and `size` to zero. These accessor methods are implemented with careful use of the `validate` and `make_position` utilities to safeguard against boundary cases.

Operations for Updating a Linked Binary Tree

Thus far, we have provided functionality for examining an existing binary tree. However, the constructor for our `LinkedBinaryTree` class results in an empty tree and we have not provided any means for changing the structure or content of a tree. We chose not to declare update methods as part of the `Tree` or `BinaryTree` abstract base classes for several reasons. First, although the principle of encapsulation suggests that the outward behaviors of a class need not depend on the internal representation, the efficiency of the operations depends greatly upon the representation. We prefer to have each concrete implementation of a tree class offer the most suitable options for updating a tree.

The second reason we do not provide update methods in the base class is that we may not want such update methods to be part of a public interface. There are many applications of trees, and some forms of update operations that are suitable for one application may be unacceptable in another. However, if we place an update method in a base class, any class that inherits from that base will inherit the update method. Consider, for example, the possibility of a method `T.replace(p, e)` that replaces the element stored at position `p` with another element `e`. Such a general method may be unacceptable in the context of an arithmetic expression tree (see Example 8.7 on page 312, and a later case study in Section 8.5), because we may want to enforce that internal nodes store only operators as elements.

Chapter 8. Trees

Performance of the Linked Binary Tree Implementation

To summarize the efficiencies of the linked structure representation, we analyze the running times of the LinkedBinaryTree methods, including derived methods that are inherited from the Tree and BinaryTree classes:

- The len method, implemented in LinkedBinaryTree, uses an instance variable storing the number of nodes of T and takes $O(1)$ time. Method is empty, inherited from Tree, relies on a single call to len and thus takes $O(1)$ time.
- The accessor methods root, left, right, parent, and num children are implemented directly in LinkedBinaryTree and take $O(1)$ time. The sibling and children methods are derived in BinaryTree based on a constant number of calls to these other accessors, so they run in $O(1)$ time as well.
- The is root and is leaf methods, from the Tree class, both run in $O(1)$ time, as is root calls root and then relies on equivalence testing of positions, while is leaf calls left and right and verifies that None is returned by both.
- Methods depth and height were each analyzed in Section 8.1.3. The depth method at position p runs in $O(dp + 1)$ time where dp is its depth; the height method on the root of the tree runs in $O(n)$ time.
- The various update methods add root, add left, add right, replace, delete, and attach (that is, their nonpublic implementations) each run in $O(1)$ time, as they involve relinking only a constant number of nodes per operation.

Table 8.1 summarizes the performance of the linked structure implementation of a binary tree.

Operation

Running Time

len, is empty

$O(1)$

root, parent, left, right, sibling, children, num children

$O(1)$

is root, is leaf

$O(1)$

depth(p)

$O(dp + 1)$

height

$O(n)$

add root, add left, add right, replace, delete, attach

$O(1)$

Table 8.1: Running times for the methods of an n-node binary tree implemented with a linked structure. The space usage is $O(n)$.

In this section, we consider a data structure known as a (2,4) tree. It is a particular example of a more general structure known as a multiway search tree, in which internal nodes may have more than two children. Other forms of multiway search trees will be discussed in Section 15.3.

11.5.1

Multiway Search Trees

Recall that general trees are defined so that internal nodes may have many children. In this section, we discuss how general trees can be used as multiway search trees. Map items stored in a search tree are pairs of the form (k, v) , where k is the key and v is the value associated with the key.

Definition of a Multiway Search Tree

Let w be a node of an ordered tree. We say that w is a d -node if w has d children.

We define a multiway search tree to be an ordered tree T that has the following properties, which are illustrated in Figure 11.23a:

- Each internal node of T has at least two children. That is, each internal node is a d -node such that $d \geq 2$.
- Each internal d -node w of T with children $c_1, \dots, c_d$ stores an ordered set of $d+1$ key-value pairs $(k_1, v_1), \dots, (k_{d+1}, v_{d+1})$, where $k_1 < \dots < k_{d+1}$.
- Let us conventionally define $k_0 = -\infty$ and $k_{d+1} = +\infty$. For each item (k, v) stored at a node in the subtree of w rooted at c_i , $i = 1, \dots, d$, we have that $k_{i-1} < k < k_i$.

That is, if we think of the set of keys stored at w as including the special fictitious keys $k_0 = -\infty$ and $k_{d+1} = +\infty$, then a key k stored in the subtree of T rooted at a child node c_i must be in between two keys stored at w . This simple viewpoint gives rise to the rule that a d -node stores $d+1$ regular keys, and it also forms the basis of the algorithm for searching in a multiway search tree.

By the above definition, the external nodes of a multiway search do not store any data and serve only as placeholders. These external nodes can be efficiently represented by None references, as has been our convention with binary search trees (Section 11.1). However, for the sake of exposition, we will discuss these as actual nodes that do not store anything. Based on this definition, there is an interesting relationship between the number of key-value pairs and the number of external nodes in a multiway search tree.

Proposition 11.7: An n -item multiway search tree has $n+1$ external nodes.

We leave the justification of this proposition as an exercise (C-11.52).